

모바일 주문과 알림, 실시간 혼잡도 예측 기능이 통합된 스마트 캠퍼스 복합 플랫폼 시스템 개발

최성호, 박정연, 최혜빈, 이다영

요 약

본 연구는 NSU 학생복지회관 식당의 운영 혼잡, 호출 사각지대, 메뉴 정보 부족 등의 문제를 해결하기 위해, Firebase 기반 실시간 데이터 동기화 아키텍처를 바탕으로 한 스마트 캠퍼스 복합 플랫폼 시스템을 설계·구현하였다. Android 기반 사용자 앱, Node.js 관리자 웹, Flask 기반 크롤링 서버, Firestore 백엔드 간의 연계를 통해 △모바일 주문 △조리 완료 실시간 알림 △혼잡도 예측 △식단 자동 수집 △광고 및 커뮤니티 기능을 통합적으로 제공한다.

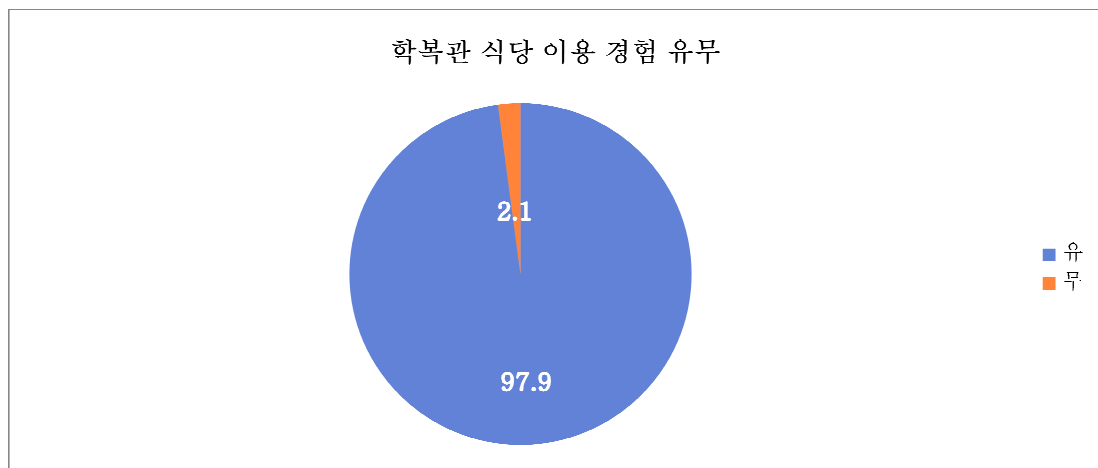
Key words: 모바일 주문, 조리 완료 실시간 알림, 혼잡도 예측 시각화, Firebase 통합 생태계, Flask + Selenium 식단 자동화

1. 서론

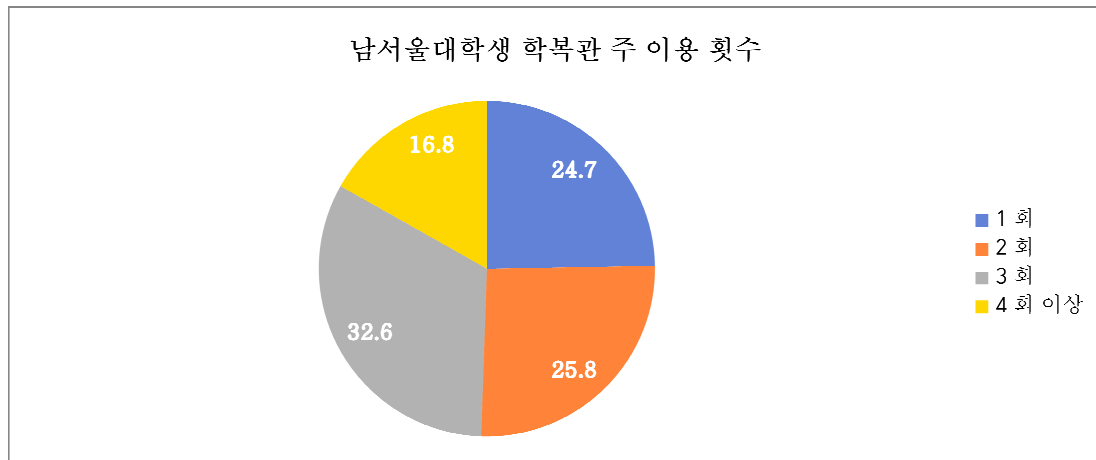
1.1 연구 배경

NSU 학생복지회관 내 식당은 하루 평균 700~800명에 달하는 높은 이용률에도 불구하고, 공간적 제약과 비효율적인 운영 시스템으로 인해 지속적으로 사용자에게 불편을 초래하고 있다. 특히 점심 시간대(11:30~13:00)에 이용이 집중되면서 키오스크(kiosk) 혼잡, 조리 완료 안내 미흡, 메뉴 정보 부족 등의 문제가 반복해서 발생하고 있다.

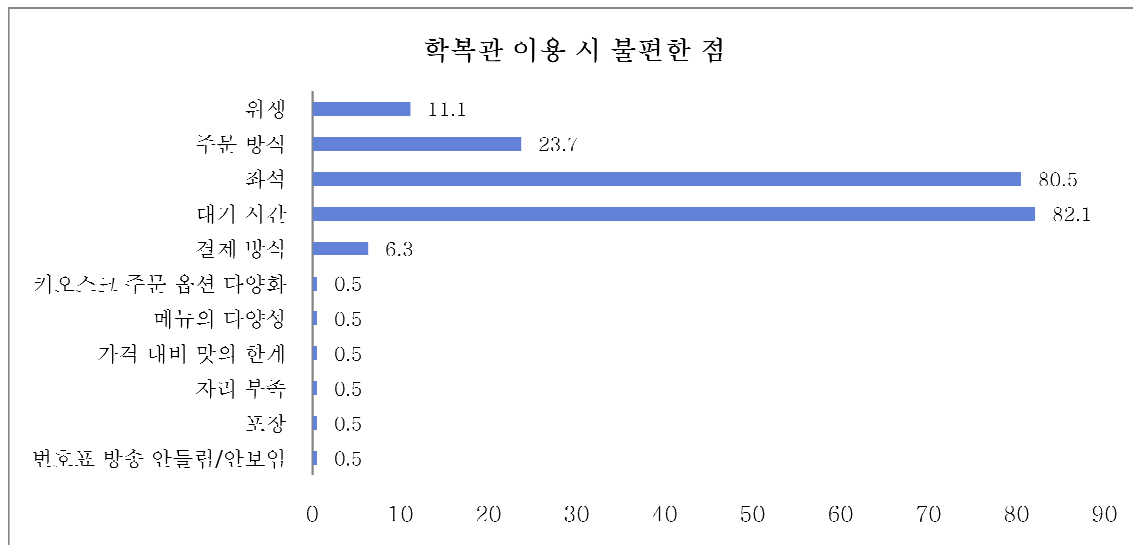
2025년 5월 학내 설문조사(응답자 190명) 결과, 전체 응답자의 81.7%가 ‘조리 대기 지연 및 호출 불편’을 경험한 것으로 나타났다. 이는 식사 포기 및 회전을 저하로 이어지는 구조적인 문제를 반영하는 결과이다.



(그림 1) 학생복지회관 실태조사 - 경험 유무



(그림 2) 학생복지회관 실태조사 - 이용 횟수



(그림 3) 학생복지회관 실태조사 - 실태조사

1.2 연구 목적

본 연구는 **학내 식당 이용자와 운영자**를 위한 **통합 스마트 플랫폼**을 설계 및 구현함으로써, **실시간 데이터 기반의 효율적인 식사 환경**을 조성하고 **사용자 편의성**을 제고하는 것을 목표로 한다. 해당 플랫폼은 “학복관 그 선배”로 명명하며, 다음과 같은 기능을 중심으로 구성된다.

1.2.1. 모바일 주문 기능

Firestore 기반 주문 저장 구조를 활용하여 **키오스크 혼잡도** 해소 및 **원격 주문** 가능.

1.2.2. 조리 완료 실시간 알림 기능

실시간 데이터베이스 **Firestore**의 상태 감지를 통해 **사용자 단말에 팝업** 표시, **호출 사각지대** 해소.

1.2.3. 식당 혼잡도 예측 기능

조리 완료 시각 로그를 기반으로 **실시간 혼잡도**를 계산하고, 시각화된 지표(UI)를 사용자에게 제공.

1.2.4. 식단 정보 제공 기능

Flask 기반 **크롤링 서버**가 **카카오 채널**에서 **식단 정보**를 **자동 수집**하여 **Firestore**에 저장, 사용자 앱에 **실시간** 표시.

1.2.5. 광고 및 커뮤니티 기능

상단 광고 배너와 **게시판** 기능을 통해 **정보 제공**과 **사용자 참여**를 유도.

1.3 연구 내용 요약

본 논문은 상기 목표 달성을 위해, 다음과 같은 내용으로 구성된다.

1.3.1. 기존 연구 사례 조사(선행연구)

관련 연구 검토를 통해 기존 테이블 오더(table order) 시스템과의 차별점을 비교하고, 캠퍼스 환경에 최적화된 기술 구성을 분석한다.

1.3.2. 시스템 아키텍처 정리, 데이터 구조

시스템 아키텍처 설계 및 데이터 구조 정의를 통해 사용자 앱(Android), 관리자 웹(Node.js), 크롤링 서버(Python Flask), Firebase와 백엔드 간의 연동 방식을 기술한다.

1.3.3. 핵심 기능별 설명, 기술적 난제 및 해결 방안

핵심 기능별 구현 내용(예: 모바일 주문, 조리 알림, 혼잡도 예측)과 기술적 난제를 설명하고, 해결 방안을 제시한다.

1.3.4. 결과 및 실증 테스트

기능별 시나리오 기반 테스트 및 실증 결과 분석을 통해 시스템 완성도와 실시간 처리 신뢰성을 검증한다.

1.3.5. 결론 및 향후 과제

본 시스템의 기대효과와 한계, 그리고 실제 식당 연동, 사용자 기반 확대, AI 추천 고도화 등 발전 방향을 제시한다.

2. 관련 연구

2.1 기존 시스템 분석

학내 식당 운영 효율성 향상을 위한 스마트 플랫폼 구축은 상용 외식업계에서 다수 도입되고 있다. 본 연구에서는 국내외 주요 테이블 오더(table order) 시스템과의 비교를 통해 기술 구조, 운영 대상, 확장성 등에서의 차이를 분석하였다.

2.1.1. 티오더(T'ORDER)

글로벌 프랜차이즈 중심의 태블릿 기반 주문 시스템으로, AI 기반 추천 시스템과 AWS 인프라를 도입하였으며, 대규모 인프라 모니터링 기능을 제공한다. 하지만 고비용 구조와 전문 유지 보수 필요로 인해 소규모 환경에는 부적합하다.

2.1.2. KT 하이오더(Hi-Order)

LTE 통신 기반 안정성, 다국어 지원, 전국 A/S 체계를 갖춘 국내 상용 테이블 오더 시스템이다. 단, 폐쇄적인 기술 구조와 초기 설치비용 부담이 높아 자체 개발·운영이 어려운 환경에는 부적합하다.

2.1.3. 메뉴잇(Menuit)

무선 오더 패드 기반의 경량 솔루션으로, 소규모 독립 매장에 적합하다. 실시간 동기화와 저비용이 장점이나, 데이터 분석 기능 미비 및 확장성 부족이 단점이다.

<표 5> 기존 테이블 오더 시스템과 본 시스템 비교표

항목	티오더 (T'ORDER)	KT 하이오더	메뉴잇 (Menuit)	학복관 그 선배
혼잡도 예측	× 없음	× 없음	× 없음	실시간 혼잡도 예측/시각화
조리 완료 알림	× 없음	× 없음	× 없음	실시간 팝업(AlertDialog)
기술 구조	AWS + AI 분석	LTE 기반 전용망	로컬 동기화 서버	Firestore + Flask + Firestore
대상 매장	프랜차이즈	일반 외식 매장	독립 소형 매장	학내 구내식당 전용
도입 환경	고비용, 전문가 유지보수	높은 초기비용	로컬 운영자 대응	Firestore 무료 티어 기반, 학생 스마트폰 활용 최적화

2.2 기술적 이론 배경

본 시스템은 Google Firebase 생태계를 기반으로 하며, 다음과 같은 주요 기술 이론을 적용하였다.

2.2.1. Firebase Firestore

NoSQL 문서형 데이터베이스(document database)로, 실시간 동기화, 계층적 경로 구조, 접근 권한 제어가 가능하다.

2.2.2. Firebase Authentication

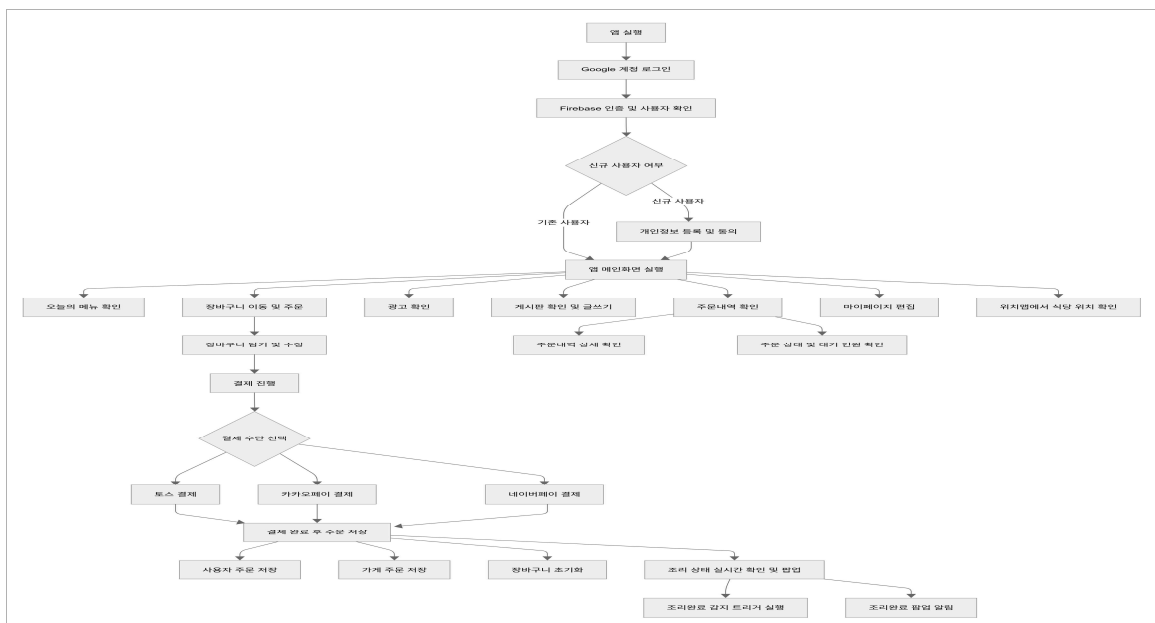
OAuth(개방형 인증) 기반의 사용자 인증 체계로, 사용자 앱과 관리자 시스템 모두에서 로그인 인증 및 접근 권한 제어를 수행한다.

2.3.2. addSnapshotListener()

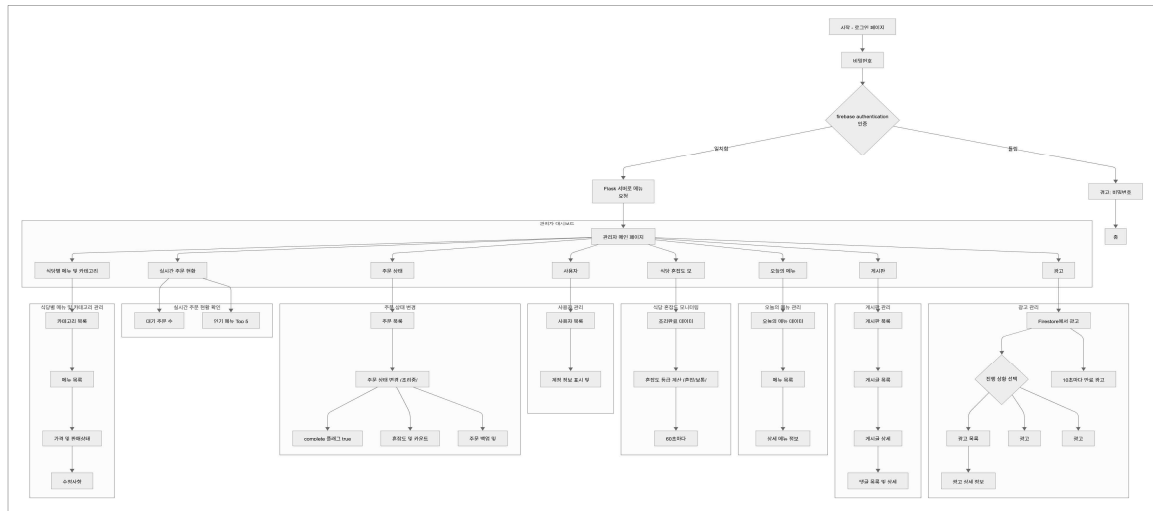
Firestore의 실시간 리스너로, 문서 변경을 감지하여 조리 완료(complete=true) 시 사용자 단말에 AlertDialog 팝업을 표시한다.

2.3.3. Flask + Selenium 기반 크롤링 서버

카카오 채널에서 식단 데이터를 수집하여, Firestore에 저장하는 백엔드 자동화 구조를 형성한다.



(그림 4) 학복관 그 선배 시스템의 앱 기술 구조 비교도



(그림 5) 학복관 그 선배 시스템의 웹 기술 구조 비교도

2.3 본 시스템의 차별성과 이론적 의의

기존 **상용 시스템**은 주로 **상업 매장을 중심**으로 설계되었으며, **고정된 단말기**, **고비용 인프라**, **데이터 예측 기능 미비** 등의 한계를 지닌다. 반면, 본 연구의 시스템은 다음과 같은 **이론적·기술적 차별성**을 갖는다.

2.3.1. 실시간성(real-time) 기반 구조

Firestore 감지, **사용자 알림**, **혼잡도 계산**이 1초 내 반영됨.

2.3.2. 감지 기반 이벤트 처리(event-driven UI)

조리 완료 → 상태 변경 → 팝업 발생 구조

2.3.3. 사용자 중심 확장성

Android MVVM(Model-View-ViewModel) 기반 구조로 **사용자별 메뉴 시각화**, **주문 UI 동기화**가 가능함.

2.3.4. Firebase 통합 생태계 활용

별도 서버 없이도 **Android 앱**, **관리자 웹**, **크롤러** 간 **실시간 연동**이 가능함.

<표 6> 본 시스템의 기술 차별성과 기존 시스템 대비 이점 요약표

비교 항목	기존 시스템	본 시스템	기술적 차별성	기대 효과
메뉴 정보 제공	관리자 수작업 입력 또는 고정형 정보 제공	Flask 기반 식단 크롤링 서버를 통 한 자동화 수집 및 실시간 업데이트	Flask + Selenium 크롤링 → Firestore 실시간 연동	관리자의 수작업 부담 감소, 사용자 최신 정보 접근성 향상
주문 및 결제	PC 키오스크 위주, 혼잡 시 대기 필수	모바일 앱을 통한 원격 주문 및 외부 간편결제 (토스/카카오/네이 버페이) 연동	모바일 최적화 UI , Firebase 기반 주문 구조	장소 구매 없는 주문 편의성, 키오스크 혼잡 해 소
조리 상태 확인	조리 완료 시 별도 안내	Firestore 실시간 감지 기반 팝업 (AlertDialog) 알림	addSnapshotList ener()로 실시간 상태 감지 및 사용자 알림	호출 사각지대 해소, 사용자 대 기 시간 최소화
혼잡도 예측	관리자 임의 판단 및 수기 공지	조리 완료 로그 기반 혼잡도 실시 간 계산 및 시각화	실시간 조리 수량 로그 기반 혼잡도 등급 계산 및 UI 반영	이용자 대기 분산 , 혼잡 피크 시간 회피 가능
광고 및 커뮤니티 기능	외부 플랫폼 활용 또는 수동 공지	Firestore 기반 광고 자동화 관리 및 앱 내 게시판 제공	TTL 기반 광고 만료 자동 이동, Firestore 게시판 관리	운영 효율성 증대 , 사용자 참여도 및 소통 활성화
시스템 구조	상용 테이블오더 시스템기반 (TORDER, KT Hi-Order 등)	Firebase 중심 통합 구조(앱, 관리자웹, 크롤러 실시간 연동)	실시간 동기화 구조, 저비용 고확장성	고비용 없이 확장 가능, 학생 중심 최적화

3. 연구 내용

3.1 요구 분석

3.1.1 기능적 요구 사항

본 시스템은 **학내 식당 운영 개선**을 목표로 하며, 다음과 같은 주요 기능이 요구되었다.

모바일 주문 기능: 키오스크 대기 문제를 해소하기 위한 **원격 주문 처리**

조리 완료 알림 기능: 호출 사각지대 해소를 위한 **실시간 팝업 알림**

혼잡도 예측 기능: 시간대별 식당 이용 집중도 시각화

오늘의 식단 자동 수집 기능: 관리자의 수동 입력 부담 제거

광고 및 게시판 기능: 정보 제공 및 커뮤니티 활성화 도모

3.1.2 비기능적 요구 사항

실시간성(real-time): 조리 완료 감지, 혼잡도 변화 등을 앱에 1초 이내 반영

확장성(scalability): 식당 수, 메뉴 수 증가에 따른 구조적 유연성 확보

보안성(security): Firebase Authentication 기반 사용자 인증, 문서별 권한 제어

접근성(accessibility): 스마트폰만으로 이용 가능, 별도 태블릿 불필요

<표 7> 본 시스템의 기술 차별성과 기존 시스템 대비 이점 요약표

구분	요구사항	상세 설명
기능적	실시간 메뉴 및 혼잡도 확인	메뉴 품질 여부, 인기 메뉴, 혼잡도 실시간 제공
기능적	원격 모바일 주문 및 간편 결제	앱을 통한 사전 주문 및 토스, 카카오페이 등 간편결제
기능적	실시간 주문 상태 및 대기 번호 확인	조리 상태, 남은 대기 인원, 예상 소요 시간 실시간 확인
기능적	좌석 현황 및 예약 기능	학복관 내 빈 좌석 현황 표시, 좌석 예약 및 시간제한 등
기능적	조리 완료 알림	앱 또는 카카오톡을 통한 조리 완료 알림
기능적	커뮤니티 및 메뉴 평가 기능	메뉴 제안, 별점 평가, 후기 작성 등 사용자 참여 활성화
기능적	음식 정보 상세 표시	칼로리, 알레르기 유발 성분 표시
비기능적	직관적이고 빠른 UI/UX	앱 내 정보 접근성, 주문/결제 빠른 처리
비기능적	접근성 및 편의성	모바일 중심 이용 환경, QR코드 활용, 전광판 좌석 표시

3.2 시스템 설계

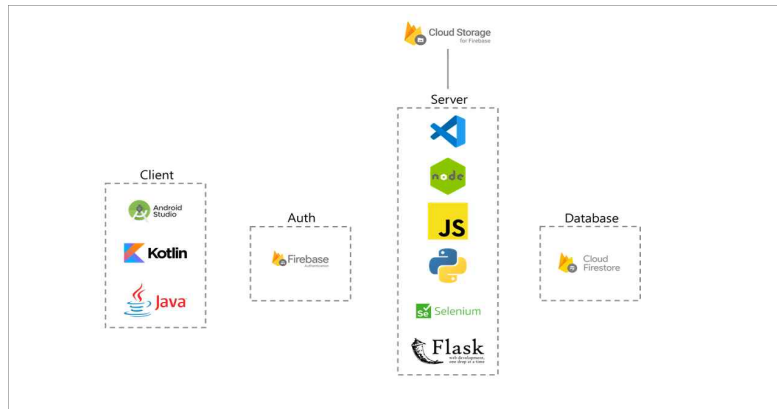
3.2.1 전체 아키텍처 구성

본 시스템은 **Android 사용자 앱**, **Node.js 기반 관리자 웹**, **Python Flask 크롤링 서버**, 그리고 **Firebase 백엔드(Firestore, Storage)**로 구성된다.

중앙 데이터베이스: Firestore를 통해 모든 모듈이 실시간 데이터 동기화

데이터 흐름 구조: 사용자 앱과 관리자 웹이 Firestore 상의 동일 문서를 기반으로 상태를 주고받음

크롤링 서버: 식단 정보 수집 후 Firestore에 자동 반영



(그림 6) 시스템 아키텍처 다이어그램

3.2.2 Firestore 데이터 구조 설계

/users/{userId}: 사용자 정보 저장 (이름, 생년월일, 전화번호)

/users/{userId}/orders/{timestamp}: 사용자별 주문 내역 저장

/users/{userId}/basket/{menuId}: 장바구니 항목 저장 (메뉴, 수량, 옵션 등)

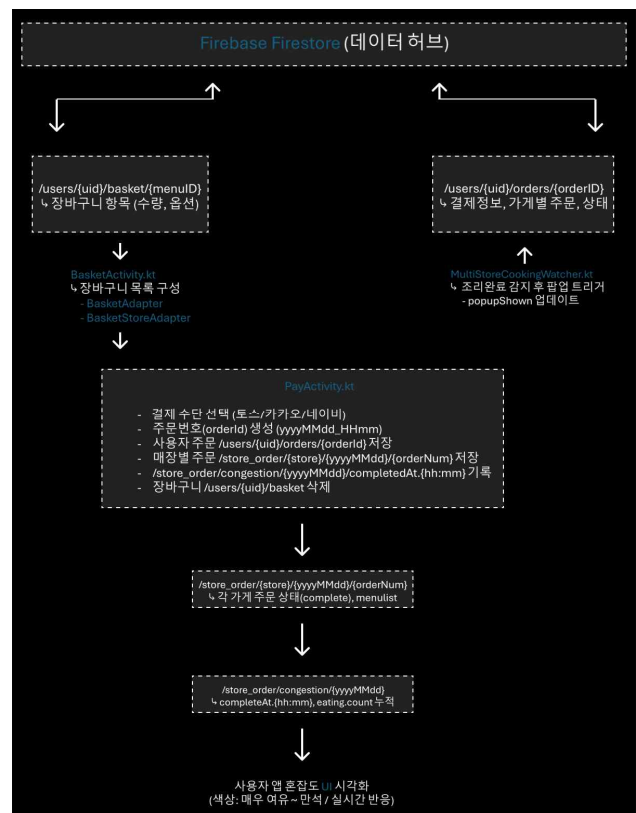
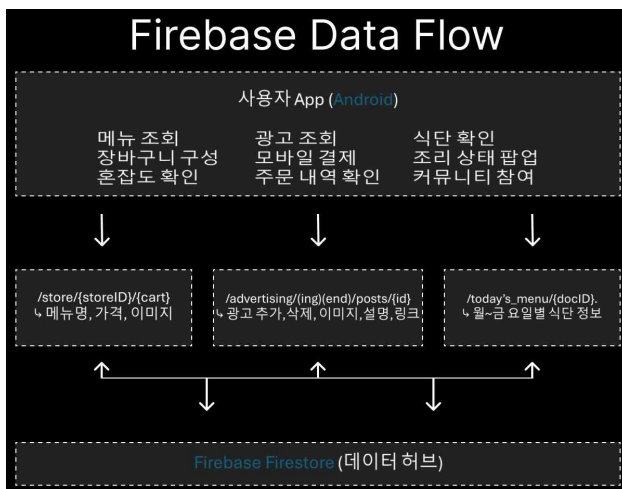
/store_order/{storeId}/{yyyyMMdd}/{orderNumber}: 매장별 조리 상태 및 주문 내역

/store_order/congestion/{yyyyMMdd}/completedAt.{hh:mm}: 시간대별 혼잡도 기록

/today's_menu/{docId}: 크롤링된 식단 정보 저장

/advertising/ing/posts/{postId}: 광고 이미지 및 설명 정보 저장

※ 장바구니(basket)는 결제 후 삭제되며, 주문 정보는 사용자와 매장 경로에 이중 저장됨.



(그림 7) Firestore 데이터베이스 구조도

<표 8> 주요 컬렉션 및 필드 요약표

컬렉션 경로	주요 필드	설명
/users/{userId}	name, birthdate, phoneNumber	사용자 기본 정보 저장 (Google 로그인 후 등록)
/users/{userId}/orders/{timestamp}	menuList, paymentMethod, complete, popupShown, store	사용자 주문 내역 및 상태 정보
/users/{userId}/basket/{menuId}	menu, amount, optionList	장바구니 메뉴 항목 저장
/store_order/{storeId}/{yyyyMMdd}/{orderId}	menuList, state, timestamp, orderId	매장별 조리 상태 및 주문 상세 정보
/store_order/congestion/{yyyyMMdd}	completedAt.hh:mm, eating.count	시간대별 혼잡도 기록 및 식사 인원 수 누적
/store/{storeId}/{category}/{menuId}	SalesStatus, imagePath, ingredient, size.price	가게별 메뉴 상세 정보 (카테고리별 구분)
/today's_menu/{docId}	Mon, Tue, Wed, Thu, Fri	요일별 식단 정보 (자동 크롤링 결과 저장)
/advertising/ing/posts/{postId}	title, imageUrl, description, redirectUrl	실시간 광고 배너에 표시될 정보
/bulletin_board/{category}/board/{postId}	title, content, voteOptions, voteDeadline, votedBy	사용자 게시글 및 투표 구성 정보
/best_menu/{storeId}/{yyyyMMdd}/menu	menu, count	금일 인기 메뉴 Top5 저장 (주문 통계 기반)

3.3 구현 기술

3.3.1 사용자 앱(Android)

언어 및 구조: Kotlin + Java 혼용, MVVM 패턴 기반 구성

3.3.1.1. 아키텍처 패턴

앱 ‘학복관 그 선배’는 핵심 기능인 ‘주문하기’를 중심으로 MVVM(Model-View-ViewModel) 아키텍처를 기반으로 설계되었다. MVVM는 Model, View, ViewModel 세 가지 구성요소를 가지며, 각각의 구성요소들이 특정한 역할을 담당하고, 서로 독립적으로 동작할 수 있게 설계된 디자인 패턴이다.

다른 디자인 패턴과 달리 MVVM 패턴은 사용자 인터페이스와 데이터 로직을 분리하며, 이를 통해, View와 ViewModel 간의 의존성이 낮아져 모듈화, 재사용성, 유지보수성 그리고 가독성이 향상된다.

반면, 실시간 데이터 처리의 중요도가 낮은 단순한 데이터 처리 기능에는 MVC(Model-View-Controller) 아키텍처를 적용했다. MVC는 Model, View, ViewModel 세 가지 구성요소를 가진다. 이 구조는 구현의 복잡도를 낮춰 기능을 효율적으로 구현할 수 있다.

메뉴 출력 기능은 실시간 데이터 반영이 필요도가 낮아서 구현의 복잡도를 낮출 수 있는 MVC 구조로 설계했다. 반면, 주문하기, 장바구니 기능은 사용자의 행동에 따라 실시간으로 데이터를 처리하여 UI에 반영해야 하므로 MVVM 아키텍처 기반으로 설계했다.

<표 9> 메뉴 출력 기능의 MVC 아키텍처 구조

구성요소	파일	기능
Model	Menu	Firestore에서 가져온 각 메뉴 데이터를 담는 클래스
View	MenuListActivity	사용자에게 메뉴 리스트를 보여주는 화면 RecyclerView와 Adapter를 통해 데이터를 리스트 형태로 시각화
Controller	MenuListActivity	View와 Model을 연결 Firestore에서 데이터를 요청하여, Menu 객체로 변환해 View에 전달

<표 10> 장바구니 기능의 MVC 아키텍처 구조

구성요소	파일	기능
Model	BasketMenu BasketRepository	Firestore에서 가져온 각 필드의 데이터를 담는 클래스 데이터베이스에서 가져온 데이터를 ViewModel로 전달
ViewModel	BasketViewModel	Model에서 받은 데이터를 가공하여 View에 전달 LiveData를 통해 Firestore의 데이터를 실시간으로 반영
View	BasketAcitivity	ViewModel에서 받은 데이터를 통해 장바구니 화면갱신 데이터 변경에 자동으로 반응하여 UI 갱신

<표 11> 주문하기 기능의 MVC 아키텍처 구조

구성요소	파일	기능
Model	OrderMenu OrderDataRepository	Firestore에서 가져온 각 필드의 데이터를 담는 클래스 데이터베이스에서 가져온 데이터를 ViewModel로 전달
ViewModel	OrderViewModel	Model에서 받은 데이터를 가공하여 View에 전달 LiveData를 통해 Firestore의 데이터를 실시간으로 반영
View	OrderHistoryAcitivity OrderDetailActivity	ViewModel에서 받은 데이터를 통해 주문내역 화면갱신 데이터 변경에 자동으로 반응하여 UI 갱신

3.3.1.2. 핵심 구현 모듈 및 흐름

BasketActivity.kt:

사용자 장바구니 화면 구현

LiveData 기반 메뉴 수량/가격 실시간 반영

BasketAdapter, BasketStoreAdapter로 가게별 그룹 구성 및 옵션 출력

PayActivity.kt:

결제 수단 선택 및 총 금액 확인

주문 데이터 가공 후 Firestore에 병렬 저장:

/users/{uid}/orders/{orderId} (사용자 확인용)

/store_order/{store}/{yyyyMMdd}/{orderNum} (매장 조리용)

결제 완료 후 장바구니 /users/{uid}/basket삭제

MultiStoreCookingWatcher.startWatching() 호출 → 조리 상태 실시간 감지 트리거

MultiStoreCookingWatcher.kt:

Firestore에서 complete=true && popupShown=false 감지 시 AlertDialog 팝업 출력

중복 팝업 방지(shownPopupStores) 및 실패 시 재시도 큐 구성

MainActivity.kt:

/advertising/ing/posts 기반 실시간 광고 슬라이더 구현
 /best_menu/{store}/{date}/menu 인기 메뉴 Top5 표시
 /store_order/congestion/{date}/eating.count 기반 혼잡도 시각화 구현

3.3.2 관리자 웹 시스템

기술 스택: HTML + JavaScript (모듈별 JS 분리) + Firebase SDK

주요 구현 기능:

hamburger_boss.js: 매장별 주문 상태 변경 및 complete=true 설정 → 사용자 앱 팝업 트리거
 todaymenu.js: 오늘의 메뉴 Firestore에서 불러오기
 run_todaymenu.py: Flask API 통해 Selenium 크롤링 실행
 advertising_add.js, advertising_del.js: 광고 이미지 Firebase Storage 업로드 및 문서 작성
 store.js: 메뉴 및 카테고리, 가격 정보 편집 UI

3.3.3 식단 크롤링 서버 (Flask + Selenium)

크롤링 대상: 카카오 채널 게시물 https://pf.kakao.com/_fghmxj/posts

파싱 구조:

게시글 제목에 '학생식당 메뉴 안내' 포함 여부 필터링
 게시글 본문 내 요일별 구분(월~금) 및 메뉴 내용 정규표현식 기반 파싱
 /today's_menu/{docId}에 Firestore 저장 (예: 2024_4월_2주차)

실행 방식:

Flask 서버의 /run-todaymenu 엔드포인트 호출 → 크롤링 실행 → JSON 결과 반환



(그림 8) 크롤링 파이프라인 구조도

4. 검증**4.1 검증 목적 및 환경**

본 시스템은 실제 식당과의 물리적 연동 없이, Firebase 실시간 데이터베이스, Android 디바이스, 웹 관리자 페이지를 활용한 모의 환경 기반 기능 테스트를 수행하였다. 검증의 목적은 다음과 같다.

각 기능별 정상 작동 여부 및 UI 반응 정확성 확인

Firestore 실시간 리스너 기반 구조의 데이터 감지 및 반영 신뢰성 평가

시뮬레이션 기반 사용자 경험 흐름 재현

<표 12> 시험 환경 구성 요약

항목	내용
사용자 앱	Kotlin 기반 Android 앱 (실디바이스/에뮬레이터)
관리자 웹	Firebase SDK 연동 Chrome 환경
백엔드	Firestore + Storage + Flask 서버
테스트 방식	시나리오 기반 단위 기능 점검
테스트 범위	주문, 조리완료 알림, 혼잡도, 광고, 식단, 투표 등

4.2 검증 방법 및 전략

단위 기능 테스트: 메뉴 선택 → 주문 저장 → 조리 완료 상태 변경 → 알림 확인

실시간 상태 시뮬레이션: 관리자 웹에서 상태 변경 시 사용자 앱에서 반응 확인

데이터 상태 기반 검증: Firestore 문서 변경 후 앱 UI 반영 여부 점검

시나리오 기반 테스트 케이스(TC) 설정 → 성공률 기록

<표 13> 검증 시나리오 흐름 요약표(TC-03, TC-06)

테스트 항목	흐름 단계	수행 주체	동작 설명
TC-03	① 주문 저장	사용자 앱	/users/{uid}/orders/{orderId}에 메뉴 및 옵션 정보 저장
	② 상태 변경	관리자 웹	/store_order/{store}/{date}/{num} 경로에서 complete = true 설정
	③ 상태 감지	사용자 앱	MultiStoreCookingWatcher.kt에서 실시간 감지
	④ 팝업 표시	사용자 앱	조건 충족 시 AlertDialog 팝업 출력, popupShown = true로 갱신
TC-06	① 혼잡도 수치 입력	관리자 웹	/store_order/congestion/{date}/completedAt.hh:mm에 값 기록 (예: 65)
	② 상태 감지 및 분류	사용자 앱	혼잡도 값 실시간 감지 후 등급(보통) 판별
	③ UI 시각화 반영	사용자 앱	혼잡도 색상 바 + 텍스트 ("보통") 동시 출력

4.3 기능별 테스트 결과

<표 14> 기능별 테스트 수행 결과 요약

기능 항목	테스트 내용	수행 결과
모바일 주문	메뉴 선택 후 Firestore 저장 여부 확인	정상 저장
조리 완료 알림	complete=true 시 팝업(AlertDialog) 표시	5건 중 5건 정상 표시
혼잡도 시각화	completedAt.{hh:mm} 입력 후 혼잡도 UI 반영	기대 색상 및 텍스트 반영
식단 정보 표시	Flask → Firestore → 앱 표시 흐름 확인	일부 예외 포맷 외 정상
광고 슬라이드	Glide 기반 이미지 순환 렌더링 확인	오류 발생 시 기본 배너 대체
투표 기능	옵션 선택 후 비율 반영 및 UI 표시 확인	시간 반영 성공

4.4 주요 테스트 사례

TC-03: 조리 완료 알림 팝업 표시

과정: 관리자 웹에서 complete=true 설정 → 앱에서 조건 감지 → 팝업 출력

결과: 5건 중 5건 정상 팝업 표시, Activity 전환 시에도 지연 감지 및 출력됨

시각화 자료: [그림 8. TC-03 조리 완료 알림 시퀀스 다이어그램]

TC-06: 혼잡도 예측 UI 반영 테스트

과정: Firestore에 혼잡도 값 입력 (예: 65) → 앱 UI에 보통(60~89) 등급 출력

결과: 색상(연노랑), 텍스트 모두 정확히 반영됨

4.5 기술적 완성도 평가

<표 15> 기술 검증 평가 항목별 종합 결과

평가 항목	평가 내용	결과
실시간 감지 정확성	complete, congestion 등 상태 감지 정확도	매우 높음 (100%)
데이터 연동 안정성	저장, 변경, 삭제 모두 정상 반영	안정적
실시간 반응 속도	관리자 조작 → 사용자 UI 반영 시간	평균 0.8초
UI 처리 신뢰성	팝업, 광고, 혼잡도 바 등	모두 정확하게 시각화됨

4.6 제약 조건 및 고려사항

<표 16> 제약 요소와 문제 처리 방식

제약 요소	처리 방식
실제 식당 장비 연동 없음	관리자 웹 수동 시뮬레이션으로 대체
실제 대규모 사용자 부하 테스트 불가	다계정 반복 시뮬레이션으로 보완
식단 크롤링 예외 상황	제목 필터 및 정규표현식 보완 예정

4.7 검증 결론

본 시스템은 실제 식당 운영 환경과 완전히 연동된 상태는 아니었으나, 기능 단위 기준으로 다음과 같은 점에서 객관적 유효성과 기술 완성도를 입증하였다.

Firestore 기반 실시간 처리 구조는 각 기능에서 1초 내 반응성을 보이며 안정적 작동을 확인했다. 또한, 사용자 앱의 UI 흐름, 관리자 웹 상태 변경 로직, 크롤링 자동화 구조는 모두 기대 수준 이상으로 동작했으며, 향후 실 환경 연동 시에도 추가 수정 없이 적용할 수 있는 기술 신뢰성을 확보했다.

5. 결론

5.1 연구 결과 요약

본 논문에서는 학내 식당 운영의 비효율성과 사용자 불편을 해결하고자, 모바일 주문, 조리 완료 알림, 혼잡도 예측, 식단 정보 자동 수집, 광고 및 커뮤니티 기능이 통합된 스마트 캠퍼스 복합 플랫폼, “학복관 그 선배”를 설계하고 구현하였다.

전체 시스템은 Firebase 실시간 데이터베이스(Firestore)를 중심으로 구성되었으며, 사용자 앱(Android), 관리자 웹 시스템(Node.js), 자동화 식단 크롤링 서버(Flask)가 실시간으로 연동되어 다음과 같은 기능을 수행하였다.

키오스크(kiosk) 혼잡 문제 해소를 위한 모바일 주문 기능

조리 상태(complete) 실시간 감지 기반 팝업(AlertDialog) 알림

시간대별 조리 완료 로그를 활용한 혼잡도(heat index) 시각화

Flask + Selenium 기반 식단 수집 자동화 및 앱 UI 연동

Firestore 공유 기반 광고·게시판 시스템을 통한 정보 제공 및 사용자 소통 활성화

5.2 기대 효과

본 시스템을 통해 다음과 같은 실질적 기대 효과를 확인하였다.

5.2.1. 사용자 측면

호출 사각지대 문제 해소 → 식사 포기율 감소
 실시간 대기 정보 확인 → 혼잡 회피 및 시간 최적화
 모바일 주문 및 결제 → 이용 편의성 향상

5.2.2. 운영자 측면

호출 실패/중복 항의 감소 → 운영상 효율 향상
 식단 정보 자동화 → 수작업 부담 경감
 인기 메뉴, 혼잡도 데이터 확보 → 운영 전략 수립 기반 제공

5.2.3. 기술적 측면

실시간 동기화 구조 완성 → 백엔드 구조 검증
 Android 앱과 웹 시스템 간 연동 → 크로스플랫폼 실현
 Firebase 중심 통합 구조 → 향후 기능 확장 용이

<표 17> 기대 효과 요약표

구분	기대 효과 내용
사용자 측면	- 호출 사각지대 해소 → 조리 완료 팝업(AlertDialog)- 실시간 혼잡도 확인 → 대기 시간 분산- 모바일 주문·결제 → 키오스크 대체
운영자 측면	- 호출 실패·중복 항의 감소- 식단 자동화 → 관리자 수동 작업 감소- 인기 메뉴·혼잡도 데이터 기반 전략 수립 가능
기술적 측면	- Firestore 실시간 리스너 기반 감지 구조 검증- Android ↔ Firebase ↔ Flask 간 연동 성공- 사용자 앱과 관리자 웹의 구조 통합으로 확장 용이성 확보

5.3 향후 연구 방향

비록 본 시스템은 기능적 완성도와 기술적 신뢰성을 갖추었으나, 다음과 같은 확장 과제 및 한계 보완 방향이 존재한다.

실제 식당 운영 환경 연동

POS(Point of Sale), 호출벨 시스템 등과의 물리적 연계 필요

현장 기반 사용자 행동 데이터(착석 시간, 회전률 등) 확보를 통한 고도화 가능

성능 및 부하 테스트

다중 동시 접속자 환경에서의 서버 반응 속도 및 처리 병목 분석

Firebase 사용량 증가 시 비용 최적화 설계 (Firestore 샤딩, 호출 주기 조절 등)

AI 기반 개인 맞춤형 추천 시스템 도입

사용자 주문 데이터를 기반으로 선호 메뉴 추천, 알레르기 필터링 등 개인화 기능 강화

Firebase MLKit 또는 외부 추천 알고리즘 연동을 통한 지능형 기능 확장

접근성과 다중 플랫폼 확대

iOS 앱, 키오스크 모드, 웹뷰 기반 버전 등 다양한 사용자 환경 대응

시각장애인·고령자 대상 접근성(Accessibility) 기능 도입

보안 강화 및 개인정보 보호 고도화

민감정보(생년월일, 전화번호 등) 암호화 저장

사용자 동의 기록, 토큰 기반 인증, 보안 로그 기록 등 강화 필요

<표 18> 향후 기능 확장 방향 개요도

구분	확장 방향 설명	기대 효과
실환경 시스템 연동	POS, 호출벨 시스템 등 식당 장비와의 물리적 연동	실제 식당 운영 흐름 반영, 현장 기반 데이터 수집 가능
성능 및 부하 대응	Firestore 샤딩, 호출 간격 조정, 동시 접속 처리 최적화	대규모 사용자 환경에서도 안정적인 운영
AI 기반 추천 기능 도입	Firebase MLKit 또는 외부 알고리즘을 활용한 개인 맞춤 메뉴 추천	사용자 만족도 향상, 건강/취향 기반 지능형 추천 구현 가능
플랫폼 및 접근성 확대	iOS 앱, 키오스크 모드, 웹뷰 포털, 음성 안내·폰트 확대 등	다양한 사용자 환경 대응, 고령자·장애인 사용자 경험 향상
보안 및 개인정보 보호 고도화	생년월일·전화번호 암호화, 동의 로그 기록, 토큰 기반 인증 등	민감정보 보호 강화, 개인정보보호법 및 기술 기준 준수

5.4 맺음말

본 연구를 통해 개발된 “학복관 그 선배” 플랫폼은 단순한 학내 식사 편의 시스템을 넘어, 실시간성, 정보 제공성, 확장 가능성을 두루 갖춘 스마트 캠퍼스 식당 운영 인프라의 선례로서 자리매김할 수 있다.

향후 실 환경 연동 및 인공지능 기술 도입을 통해, 보다 지능적이고 지속 가능한 학내 복지 서비스 시스템으로 발전할 수 있을 것으로 기대된다.

참고문헌

- [1] Firebase Firestore 공식 문서
<https://firebase.google.com/docs/firestore>
- [2] Firebase Authentication 공식 문서
<https://firebase.google.com/docs/auth>
- [3] Firebase Storage 공식 문서
<https://firebase.google.com/docs/storage>
- [4] Google Sign-In for Android 개발자 문서
<https://developers.google.com/identity/sign-in/android/start>
- [5] Glide Android 이미지 로딩 라이브러리
<https://github.com/bumptech/glide>
- [6] Flask 공식 문서 (REST API 설계 참조)
<https://flask.palletsprojects.com/>
- [7] Selenium with Python 공식 문서
<https://selenium-python.readthedocs.io/>
- [8] NIST SP 800-63B. Digital Identity Guidelines (2017)
<https://pages.nist.gov/800-63-3/sp800-63b.html>
- [9] 학복관 그 선배 사용자 앱 기술 명세서
(PDF) 사용자 앱 기능 정의 및 구조, 실시간 감지 모듈 등 기술 설명서, 2025.05
- [10] 학복관 그 선배 관리자 웹 시스템 기술 명세서
(PDF) 광고 관리, 조리 상태 변경, 식단 크롤링 서버 기술 정의서, 2025.05
- [11] 안드로이드 스튜디오 공식 문서
<https://developer.android.com/develop/>

————— [저자 소개] —————

최성호



박정연



최혜빈



이다영

